RELATÓRIO DE SEGURANÇA

Relatório de Auditoria de Segurança — Yield

Data: **30/10/2026** · Base auditada: **7bb9c48 (main)**

Escopo auditado

Repositório completo: `src/` (106 arquivos TS/TSX), `src-tauri/` (core Rust, 17 migrações, capabilities, tauri.conf.json), `.github/workflows/`, docs, histórico git (224 commits) e o bundle `dist/`. Cinco categorias: isolamento de dados, permissão decidida no cliente, IDOR, chaves expostas e XSS.

Stack detectada

Linguagens	TypeScript (frontend), Rust 2024 (core)
Framework	Tauri 2 (app desktop macOS/Windows) + React 19 + Vite 8
Banco	SQLite local via tauri-plugin-sql, SQL escrito à mão com placeholders ?
Auth / segredos	Sem login. Credenciais Pluggy e Binance no cofre do SO (keyring), backup com AES-256-GCM + Argon2id
Rede	tauri-plugin-http com allowlist de 5 hosts (Yahoo, Pluggy, BCB, Binance)
Deploy	GitHub Actions (build.yml, release.yml) + updater do Tauri assinado (minisign)

Nota metodológica

O Yield não tem servidor nem contas de usuário: é um app desktop em que o webview (React) conversa com o core (Rust) por IPC. A fronteira de confiança que importa é webview → core. Por isso cada categoria foi traduzida para o equivalente desta stack:

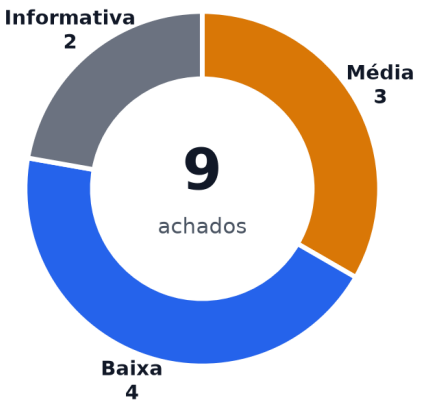
#	Categoria	Como foi mapeada nesta stack
1	**Banco sem tranca**	App desktop de usuário único: não há tenant, login nem RLS. O isolamento equivalente é (a) as capabilities do Tauri, que dizem o que o webview pode chamar, e (b) a proteção do arquivo SQLite e das cópias no disco.
2	**Permissão definida no navegador**	Não existem papéis (isAdmin, canEdit). Os "Módulos" de Configurações só escondem itens do menu e não são controle de acesso. O equivalente é o core Rust confiar no que o webview pede: segredos devolvidos ao JS e comandos privilegiados sem checagem.
3	**IDOR**	Não há objetos de outros usuários. O equivalente são os comandos IPC (#[tauri::command]) que recebem um caminho de arquivo e agem sobre ele sem conferir se está no escopo do app. Os 8 comandos registrados foram lidos um a um.
4	**Chaves expostas**	Busca por padrões de segredo no código, configs, workflows de CI, docs, histórico git completo (224 commits, todas as branches) e no bundle dist/. Não há docker-compose, Helm nem Terraform.
5	**Inputs sem tratamento (XSS)**	Frontend React 19: busca por dangerouslySetInnerHTML, innerHTML, eval, new Function, href/src dinâmicos e window.open; revisão da CSP do Tauri e das libs que parseiam arquivos do usuário (pdf.js, SheetJS). Não há backend HTTP nem e-mails.

Só entram achados verificados no código. Cada um traz arquivo e linha, trecho, por que é explorável e a condição de exploração. Escala: **crítica** (explorável direto, dano grave), **alta**, **média** (dano grave mas precisa de uma condição extra), **baixa** (impacto ou probabilidade pequenos) e **informativa** (decisão de projeto registrada para contexto).

Resumo executivo

Nenhum achado crítico ou alto. O código básico está sólido: SQL parametrizado, nenhum sink de XSS, segredos fora do repositório e no cofre do SO, backup com criptografia correta. Os riscos que sobram se concentram na fronteira webview → core: comandos de arquivo sem escopo e CSP desligada. Isolados, exigem que algum JavaScript estranho rode no webview; juntos, um XSS futuro viraria apagar arquivos do usuário e ler suas credenciais bancárias.

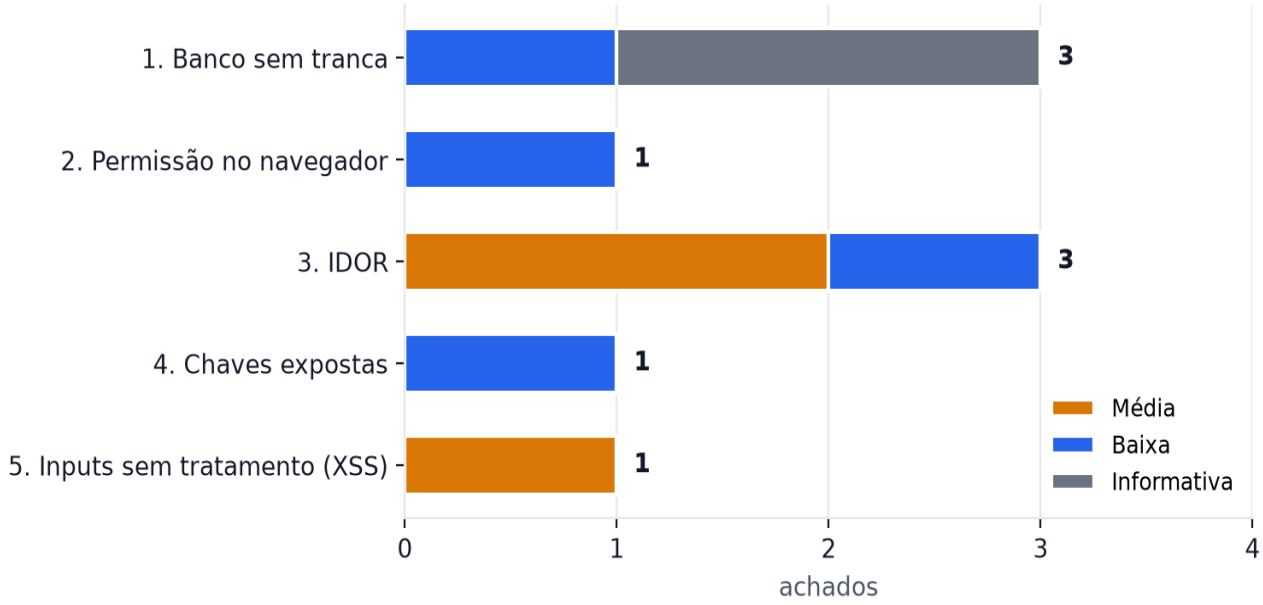
0	0	3	4	2	11
Crítica	Alta	Média	Baixa	Informativa	Pontos fortes



Achados por severidade

- Crítica: **0**
- Alta: **0**
- Média: **3**
- Baixa: **4**
- Informativa: **2**
- Pontos fortes: **11**

Achados por categoria



Pontos fortes

O que foi verificado e está correto, com a evidência.

- **SQL parametrizado em todo o código**
  Todas as consultas de src/db, src/domain, src/import e src/openfinance usam placeholders ?. O único identificador interpolado vem de lista fixa: src/db/gastosRepo.ts:130 (['category_overrides', 'category_rules', 'budget_lines']).
- **Nenhum sink de XSS no frontend**
  Zero ocorrências de dangerouslySetInnerHTML, innerHTML, outerHTML, insertAdjacentHTML, document.write, eval, new Function, window.open, <a href>/<img src> dinâmicos em src/. Todo texto passa pelo escape do JSX.
- **Libs de parsing atualizadas**
  pdfjs-dist 6.4.299 (fora da CVE-2024-4367, corrigida na 4.2.67) e SheetJS 0.20.3 (corrige CVE-2023-30533 e CVE-2024-22363).
- **Rede com allowlist em duas camadas**
  capabilities/default.json:13-32 libera só 5 hosts; src/openfinance/pluggy.ts:30-32 e src/cripto/binance.ts:18-19 recusam qualquer método/caminho fora da lista (só leitura).
- **Erros de API não vazam segredo**
  Mensagens de erro da Pluggy/Binance não incluem corpo nem chave; coberto por src/openfinance/pluggy.test.ts:158-162.
- **Cofre do SO com allowlist de chaves**
  Credenciais só no keyring (src-tauri/src/lib.rs:134-177); entry_for() recusa chave fora de SECRET_KEYS, com teste em lib.rs:347.
- **Backup bem criptografado**
  AES-256-GCM com chave Argon2id, sal de 16 bytes e nonce novos a cada backup (lib.rs:54-82); senha mínima de 8 caracteres (src/pages/configuracoes/build.ts:174); teste de senha errada e adulteração em lib.rs:313.
- **Nenhum segredo no repositório**
  Código, configs, docs, histórico git (224 commits, todas as branches) e bundle dist/ limpos. Credenciais dos testes são falsas ('cid', 'segredo-xyz'). Chave pública do updater e fingerprint do certificado (release.yml:20) são públicos por natureza.
- **Segredos de CI via secrets.***
  release.yml:43-44 e 58-60 leem tudo de secrets; o .p12 é apagado após importar (release.yml:52) num keychain temporário.
- **Atualizações assinadas**
  Updater só aceita pacotes assinados com a chave minisign (tauri.conf.json:42) e baixa de HTTPS no GitHub.
- **Superfície de plugins enxuta**
  Sem plugin de fs nem shell; capability só para a janela main. Dados pessoais (planilhas/, notas/, extratos/) estão no .gitignore e as fixtures de teste são sintéticas.

Pontos fracos

Os riscos centrais, do mais para o menos grave.

MÉDIA	**Comandos de arquivo sem escopo (F1, F2, F3)** encrypt_backup apaga qualquer arquivo depois de cifrá-lo; move_file move qualquer arquivo e nem é usado.
MÉDIA	**CSP desligada (F4)** Não há segunda barreira: um XSS futuro herdaria todos os comandos IPC.
BAIXA	**Segredos entregues ao JS (F5)** clientSecret da Pluggy e secret da Binance chegam em texto puro ao webview.
BAIXA	**Backup descriptografado esquecido (F6)** restore.db fica no disco sem senha quando a importação é cancelada.
BAIXA	**Supply chain do release (F7)** A chave do updater é passada a actions referenciadas por tag mutável.

Achados detalhados

Tabela por categoria e, em seguida, a ficha de cada achado com trecho do código.

1. Banco sem tranca

Severidade	Arquivo:linha	Descrição
BAIXA	`src-tauri/src/lib.rs:102-109, 38-40; src/pages/Configuracoes.tsx:156`	**F6** · Backup com senha fica descriptografado no disco (restore.db) ao cancelar a importação
INFORMATIVA	`src-tauri/capabilities/default.json:9-12; src/pages/Configuracoes.tsx:59`	**F8** · Webview tem SQL irrestrito e pode abrir qualquer arquivo SQLite
INFORMATIVA	`src-tauri/src/lib.rs:13-14`	**F9** · Banco e cópias de segurança ficam em texto puro e se acumulam

2. Permissão no navegador

Severidade	Arquivo:linha	Descrição
BAIXA	`src-tauri/src/lib.rs:164-167; src/secrets/vault.ts:12,18-19; src/cripto/binance.ts:34-35`	**F5** · Segredos da Pluggy e da Binance são devolvidos em texto puro ao JavaScript

3. IDOR

Severidade	Arquivo:linha	Descrição
MÉDIA	`src-tauri/src/lib.rs:84-91 (registro na 303)`	**F1** · encrypt_backup lê, criptografa e APAGA qualquer arquivo indicado pelo webview
MÉDIA	`src-tauri/src/lib.rs:44-52 (registro na 303)`	**F2** · move_file move/sobrescreve qualquer arquivo e não é usado pelo app
BAIXA	`src-tauri/src/lib.rs:33-42, 93-99, 102-109`	**F3** · swap_database, is_encrypted_backup e decrypt_backup aceitam qualquer caminho

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
BAIXA	`.github/workflows/release.yml:25-39, 56-60`	**F7** · Chave de assinatura do updater exposta a actions de terceiros referenciadas por tag mutável

5. Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição
MÉDIA	`src-tauri/tauri.conf.json:21-24`	**F4** · CSP desligada na janela do app

Fichas dos achados

MÉDIA

F1 · encrypt_backup lê, criptografa e APAGA qualquer arquivo indicado pelo webview

Categoria: 3. IDOR · **Local:** `src-tauri/src/lib.rs:84-91 (registro na 303)`

```
#[tauri::command]
fn encrypt_backup(from: String, to: String, password: String) -> Result<(), String> {
    let plain = std::fs::read(&from).map_err(|e| e.to_string())?;
    let sealed = seal(&plain, &password);
    std::fs::remove_file(&from).map_err(|e| e.to_string())?;
    std::fs::write(&to, sealed?).map_err(|e| e.to_string())
}
```

Por que é explorável: from e to vêm do JS sem nenhuma checagem de pasta ou extensão. Um script no webview pode chamar invoke('encrypt_backup', {from: '~/Documents/x.pdf', to: '/tmp/y', password: 'p'}): o arquivo original é apagado e fica só uma cópia cifrada com a senha do atacante (primitiva de ransomware/destruição de qualquer arquivo que o usuário pode escrever).

Condição de exploração: Precisa de JavaScript arbitrário rodando no webview (XSS ou dependência npm comprometida). Nenhum sink de XSS foi encontrado hoje, mas a CSP está desligada (F4).

MÉDIA

F2 · move_file move/sobrescreve qualquer arquivo e não é usado pelo app

Categoria: 3. IDOR · **Local:** `src-tauri/src/lib.rs:44-52 (registro na 303)`

```
#[tauri::command]
fn move_file(from: String, to: String) -> Result<(), String> {
    if std::fs::rename(&from, &to).is_ok() {
        return Ok(());
    }
    std::fs::copy(&from, &to).map_err(|e| e.to_string())?;
    std::fs::remove_file(&from).map_err(|e| e.to_string())
}
```

Por que é explorável: Nenhum invoke('move_file') existe em src/ (grep), mas o comando continua registrado no invoke_handler e aceita quaisquer caminhos. Permite mover, sobrescrever ou sumir com arquivos do usuário. É superfície de ataque sem uso.

Condição de exploração: Mesma de F1: JS arbitrário no webview.

MÉDIA

F4 · CSP desligada na janela do app

Categoria: 5. Inputs sem tratamento (XSS) · **Local:** `src-tauri/tauri.conf.json:21-24`

```
"security": {
  "csp": null
}
```

Por que é explorável: Sem CSP não há segunda barreira: se algum dado importado (extrato OFX/CSV, planilha B3, PDF de nota, resposta da Pluggy/Binance) virar HTML executável no futuro, o script ganha acesso a todos os comandos IPC: lê as credenciais (F5), apaga arquivos (F1/F2) e roda SQL no banco (F8). Hoje o React escapa tudo e não há sink perigoso.

Condição de exploração: Agravante: vira explorável junto com um XSS ou dependência comprometida. Desde o commit efa2177 (26/10/2026).

BAIXA

F3 · swap_database, is_encrypted_backup e decrypt_backup aceitam qualquer caminho

Categoria: 3. IDOR · **Local:** `src-tauri/src/lib.rs:33-42, 93-99, 102-109`

```
fn swap_database(app: tauri::AppHandle, source: String) -> Result<String, String> {
    let dir = app.path().app_config_dir().map_err(|e| e.to_string())?;
    let source = Path::new(&source);
    let backup = swap_in_dir(&dir, source)?;
```

Por que é explorável: swap_database copia qualquer arquivo por cima do yield.db (o banco anterior vira seguranca-<ts>.db, então dá para desfazer). decrypt_backup e is_encrypted_backup leem qualquer caminho, mas só revelam se o arquivo começa com YIELDBK1; unseal() recusa o resto. Impacto limitado a negação de serviço do app.

Condição de exploração: JS arbitrário no webview.

BAIXA

F5 · Segredos da Pluggy e da Binance são devolvidos em texto puro ao JavaScript

Categoria: 2. Permissão no navegador · **Local:** `src-tauri/src/lib.rs:164-167; src/secrets/vault.ts:12,18-19; src/cripto/binance.ts:34-35`

```
#[tauri::command]
fn secret_get(key: String) -> Result<Option<String>, String> {
    get_with(&entry_for(&key)?)
}
// binance.ts:34
const u = binanceRequest('GET', `${path}?${query}&signature=${await sign(creds.secret, query)}`)
```

Por que é explorável: O cofre do SO protege o segredo em repouso, mas o core entrega o JSON com clientSecret da Pluggy e a secret key da Binance ao webview, que assina o HMAC em JS. Qualquer código que rode no webview lê os dois e acessa todos os dados bancários do usuário via Pluggy.

Condição de exploração: JS arbitrário no webview. Por design hoje; o risco cresce com F4.

BAIXA

F6 · Backup com senha fica descriptografado no disco (restore.db) ao cancelar a importação

Categoria: 1. Banco sem tranca · **Local:** `src-tauri/src/lib.rs:102-109, 38-40; src/pages/Configuracoes.tsx:156`

```
let out = dir.join(RESTORE_FILE);
std::fs::write(&out, plain).map_err(|e| e.to_string())?;
// Configuracoes.tsx:156
<button ... onClick={() => setRestore(null)}>Cancelar</button>
```

Por que é explorável: decrypt_backup grava o banco inteiro em texto puro em <app_config_dir>/restore.db. Só swap_database apaga esse arquivo. Se o usuário cancela (ou a validação falha), a cópia sem senha fica lá para sempre, anulando a proteção do backup .yield.

Condição de exploração: Acesso de leitura à pasta do app (outro usuário/malware local, backup do Time Machine, sincronização de nuvem).

BAIXA F7 · Chave de assinatura do updater exposta a actions de terceiros referenciadas por tag mutável

Categoria: 4. Chaves expostas · **Local:** `.github/workflows/release.yml:25-39, 56-60`

```
- uses: tauri-apps/tauri-action@v0
  env:
    TAURI_SIGNING_PRIVATE_KEY: ${{ secrets.TAURI_SIGNING_PRIVATE_KEY }}
    TAURI_SIGNING_PRIVATE_KEY_PASSWORD: ${{ secrets.TAURI_SIGNING_PRIVATE_KEY_PASSWORD }}
```

Por que é explorável: Os segredos não estão no código (correto), mas tauri-action@v0, swatinem/rust-cache@v2, pnpm/action-setup@v4 e dtolnay/rust-toolchain@stable são tags que o dono da action pode mover. Uma tag sequestrada roda no job que tem a chave privada do updater: o atacante assina uma atualização maliciosa que todos os apps instalados aceitam.

Condição de exploração: Comprometimento de uma action de terceiros (supply chain). Probabilidade baixa, impacto alto.

INFORMATIVA F8 · Webview tem SQL irrestrito e pode abrir qualquer arquivo SQLite

Categoria: 1. Banco sem tranca · **Local:** `src-tauri/capabilities/default.json:9-12; src/pages/Configuracoes.tsx:59`

```
"sql:default",
"sql:allow-execute",
"sql:allow-close",
// Configuracoes.tsx:59
source = await openDb(`sqlite:${path}`)
```

Por que é explorável: É o modelo do app (todo o domínio roda em TS), não um furo isolado: o webview lê e escreve o banco inteiro e o load do plugin aceita qualquer caminho. Com ATTACH/VACUUM INTO o SQL também cria arquivos SQLite em qualquer pasta. Fica registrado porque define o tamanho do estrago de um XSS.

Condição de exploração: JS arbitrário no webview.

INFORMATIVA F9 · Banco e cópias de segurança ficam em texto puro e se acumulam

Categoria: 1. Banco sem tranca · **Local:** `src-tauri/src/lib.rs:13-14`

```
let backup = dir.join(format!("seguranca-{secs}.db"));
std::fs::copy(&db, &backup).map_err(err)?;
```

Por que é explorável: yield.db não é cifrado em repouso (decisão de projeto) e cada importação de backup deixa mais uma cópia seguranca-<ts>.db completa, sem rotação.

Condição de exploração: Acesso de leitura à pasta do app.

Recomendações priorizadas

Prioridade	Ação
P1 MÉDIA	**Escopar os comandos IPC de arquivo (F1, F2, F3)** Apagar move_file. Em encrypt_backup aceitar só from dentro do app_data_dir com prefixo export-. Abrir os diálogos de salvar/abrir no Rust (plugin dialog) para que o caminho de destino e o de origem nunca venham do JS.
P2 MÉDIA	**Ligar a CSP (F4)** Definir app.security.csp com default-src 'self', script-src 'self', connect-src ipc: http://ipc.localhost, worker-src 'self' blob: e img-src 'self' data:. Testar import de PDF e gráficos depois.
P3 BAIXA	**Tirar os segredos do JavaScript (F5)** Mover POST /auth da Pluggy e a assinatura HMAC da Binance para comandos Rust; o JS recebe só o token de 2h ou a resposta.
P4 BAIXA	**Limpar restore.db (F6)** Apagar restore.db ao cancelar, quando a validação falha e no startup do app.
P5 BAIXA	**Fixar as actions por SHA (F7)** Trocar @v0/@v2/@v4/@stable pelo SHA do commit e proteger o job de release com um environment com aprovação.
P6 INFORMATIVA	**Dados em repouso (F8, F9)** Rotacionar as cópias seguranca-*.db (manter as N últimas) e avaliar SQLCipher se o app passar a guardar mais dados sensíveis.

F8 e F9 são informativas (decisões de projeto) e não viraram issue; ficam como contexto para P6.

ISSUES PARA O GITHUB

5 issues prontas para copiar e colar. F1, F2 e F3 foram agrupados numa issue só (mesma causa e mesma correção). Copie o texto entre os marcadores.

MÉDIA **Issue 1 · [Segurança] Comandos IPC de arquivo aceitam qualquer caminho (encrypt_backup, move_file, swap_database)**

````
--- ISSUE 1 ---
# [Segurança] Comandos IPC de arquivo aceitam qualquer caminho (encrypt_backup, move_file, swap_database)

**Labels sugeridas:** security, severity:media

## Descrição
Os comandos Rust que mexem em arquivos recebem caminhos vindos do webview e não conferem se estão dentro
da pasta do app. Qualquer JavaScript que rode no webview (XSS ou dependência npm comprometida) pode
apagar, mover ou sobrescrever arquivos do usuário. `encrypt_backup` ainda apaga o original depois de
cifrá-lo com uma senha escolhida pelo chamador.

## Evidência
`src-tauri/src/lib.rs:84-91`
```rust
fn encrypt_backup(from: String, to: String, password: String) -> Result<(), String> {
    let plain = std::fs::read(&from).map_err(|e| e.to_string())?;
    let sealed = seal(&plain, &password);
    std::fs::remove_file(&from).map_err(|e| e.to_string())?;
    std::fs::write(&to, sealed?).map_err(|e| e.to_string())
}
```
`src-tauri/src/lib.rs:44-52` - `move_file(from, to)` faz rename/copy+remove de qualquer caminho e não tem
nenhum `invoke('move_file')` em `src/`.
`src-tauri/src/lib.rs:33-42` - `swap_database(source)` copia qualquer arquivo por cima do `yield.db`.
`src-tauri/src/lib.rs:303` - os três estão no `invoke_handler`.

## Impacto
Com JS no webview: destruição ou "sequestro" de qualquer arquivo que o usuário pode escrever, e corrupção
do banco do app.

## Sugestão de correção
- Remover `move_file` do código e do `generate_handler!`.
- `encrypt_backup`: aceitar só `from` dentro de `app_data_dir` com nome `export-*.db`.
- Fazer os diálogos de salvar/abrir no Rust e não receber `to`/`source` do JS.
- `swap_database`: aceitar só `restore.db` ou um caminho devolvido pelo diálogo nativo.

## Critérios de aceite
- [ ] `move_file` não existe mais no `lib.rs`.
- [ ] `invoke('encrypt_backup', { from: '<arquivo fora do app>' , ... })` devolve erro e não apaga nada.
- [ ] `swap_database` com caminho arbitrário devolve erro.
- [ ] Teste Rust cobrindo a recusa de caminhos fora do escopo.
- [ ] Exportar e importar backup continuam funcionando.
--- FIM ISSUE 1 ---
````

MÉDIA **Issue 2 · [Segurança] CSP desligada na janela do Tauri**

````
--- ISSUE 2 ---
# [Segurança] CSP desligada na janela do Tauri

**Labels sugeridas:** security, severity:media

## Descrição
A janela do app roda sem Content Security Policy. Hoje não há sink de XSS (o React escapa tudo), mas o app
processa muitos dados de fora (OFX/CSV, planilhas da B3, PDFs de notas, respostas da Pluggy e da Binance).
Sem CSP, um único XSS no futuro tem acesso total aos comandos IPC: credenciais, arquivos e banco.

## Evidência
`src-tauri/tauri.conf.json:21-24`
```json
"security": {
  "csp": null
}
```

## Impacto
Transforma qualquer XSS futuro em leitura das credenciais da Pluggy/Binance, destruição de arquivos (ver
issue dos comandos IPC) e SQL arbitrário no banco.

## Sugestão de correção
```json
"csp": "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:
blob:; worker-src 'self' blob:; connect-src ipc: http://ipc.localhost"
```
Ajustar após testar import de PDF (worker do pdf.js) e as telas com gráficos.

## Critérios de aceite
- [ ] `app.security.csp` definido no `tauri.conf.json` sem `'unsafe-eval'`.
- [ ] Build de release abre todas as telas sem erro de CSP no console.
- [ ] Importação de PDF, B3, OFX/CSV, sync da Pluggy e da Binance funcionam.
--- FIM ISSUE 2 ---
````

BAIXA **Issue 3 · [Segurança] Segredos da Pluggy e da Binance chegam em texto puro ao JavaScript**

```
--- ISSUE 3 ---
# [Segurança] Segredos da Pluggy e da Binance chegam em texto puro ao JavaScript

**Labels sugeridas:** security, severity:baixa

## Descrição
O cofre do SO protege as credenciais em repouso, mas `secret_get` devolve o JSON completo ao webview, que
usa o `clientSecret` da Pluggy para o `POST /auth` e a secret da Binance para assinar o HMAC em JS.
Qualquer script no webview lê as duas.

## Evidência
`src-tauri/src/lib.rs:164-167`
```rust
fn secret_get(key: String) -> Result<Option<String>, String> {
    get_with(&entry_for(&key)?)
}
```
`src/secrets/vault.ts:12,18-19` lê e faz `JSON.parse` do item. `src/cripto/binance.ts:34` assina com
`creds.secret`.

Impacto
Com JS no webview: acesso de leitura a todas as contas e investimentos ligados na Pluggy e aos
saldos/histórico da Binance.

Sugestão de correção
- Criar comandos Rust `pluggy_auth` (devolve só a apiKey de 2h) e `binance_signed_get(path, query)`.
- `secret_get` passa a devolver só se a credencial existe (bool).

Critérios de aceite
- [ ] Nenhum `clientSecret` nem secret da Binance trafega do Rust para o JS.
- [ ] `secret_get` não devolve o valor do segredo.
- [ ] Sync da Pluggy e da Binance continuam funcionando, com teste.
--- FIM ISSUE 3 ---
```

BAIXA **Issue 4 · [Segurança] Backup com senha fica descriptografado no disco (restore.db) ao cancelar**

```
--- ISSUE 4 ---
# [Segurança] Backup com senha fica descriptografado no disco (restore.db) ao cancelar

**Labels sugeridas:** security, severity:baixa

## Descrição
Ao abrir um backup `.yield`, `decrypt_backup` grava o banco inteiro em texto puro em
`<app_config_dir>/restore.db`. Só `swap_database` apaga esse arquivo. Se o usuário clica em Cancelar ou a
validação falha, a cópia sem senha fica no disco.

## Evidência
`src-tauri/src/lib.rs:106-107`
```rust
let out = dir.join(RESTORE_FILE);
std::fs::write(&out, plain).map_err(|e| e.to_string())?;
```
`src-tauri/src/lib.rs:38-40` - único ponto que remove o arquivo.
`src/pages/Configuracoes.tsx:156` - Cancelar só faz `setRestore(null)`.

Impacto
Anula a criptografia do backup para quem lê a pasta do app (malware local, outro usuário, Time Machine,
pasta sincronizada).

Sugestão de correção
- Comando `discard_restore` chamado no Cancelar e quando `validar` falha.
- Apagar `restore.db` no startup do app.

Critérios de aceite
- [ ] Depois de Cancelar, `restore.db` não existe.
- [ ] Depois de uma validação que falha, `restore.db` não existe.
- [ ] Ao abrir o app, um `restore.db` órfão é apagado.
--- FIM ISSUE 4 ---
```

BAIXA **Issue 5 · [Segurança] Fixar por SHA as actions que recebem a chave de assinatura do updater**

````
--- ISSUE 5 ---
# [Segurança] Fixar por SHA as actions que recebem a chave de assinatura do updater

**Labels sugeridas:** security, severity:baixa

## Descrição
O job de release passa `TAURI_SIGNING_PRIVATE_KEY` para `tauri-apps/tauri-action@v0` e roda no mesmo job
`pnpm/action-setup@v4`, `swatinem/rust-cache@v2` e `dtolnay/rust-toolchain@stable`. Tags podem ser
movidas pelo dono da action; uma tag sequestrada recebe a chave.

## Evidência
`.github/workflows/release.yml:56-60`
```yaml
- uses: tauri-apps/tauri-action@v0
  env:
    TAURI_SIGNING_PRIVATE_KEY: ${{ secrets.TAURI_SIGNING_PRIVATE_KEY }}
    TAURI_SIGNING_PRIVATE_KEY_PASSWORD: ${{ secrets.TAURI_SIGNING_PRIVATE_KEY_PASSWORD }}
```
Linhas 25-39: demais actions por tag.

## Impacto
Com a chave do updater, um atacante publica uma atualização maliciosa aceita por todos os apps instalados.

## Sugestão de correção
- Trocar cada `@tag` pelo SHA completo do commit (com comentário da versão).
- Ativar Dependabot para `github-actions`.
- Colocar o job de release num `environment` com aprovação manual.

## Critérios de aceite
- [ ] Todas as `uses:` de `release.yml` e `build.yml` usam SHA de 40 caracteres.
- [ ] Dependabot configurado para GitHub Actions.
- [ ] Release só roda após aprovação no environment.
--- FIM ISSUE 5 ---
````